

构建后处理与脚本编译符号 (Build PostProcessing & Scripting Define Symbols)

后处理：iOS

在构建后处理过程中：

- 添加框架：
 - `CoreMIDI.framework`
 - `CoreAudioKit.framework`
- 修改 `Info.plist`：
 - 添加 `NSBluetoothAlwaysUsageDescription`

后处理：Android

在构建后处理过程中：

- 修改 `AndroidManifest.xml` 并添加以下权限：
 - `android.permission.BLUETOOTH`
 - `android.permission.BLUETOOTH_ADMIN`
 - `android.permission.ACCESS_FINE_LOCATION`
 - `android.permission.BLUETOOTH_SCAN`
 - `android.permission.BLUETOOTH_CONNECT`
 - `android.permission.BLUETOOTH_ADVERTISE`
- 添加所需功能：
 - `android.hardware.bluetooth_le`
 - `android.hardware.usb.host`

Meta Quest (Oculus Quest): USB MIDI 设备检测

如果你想在 Meta Quest 设备上使用 USB MIDI，必须在后处理期间启用 USB intent filter（意图过滤器）。

在 `PostProcessBuild.cs` 中，取消注释添加 Oculus USB 意图过滤器的行：

```
// androidManifest.AddUsbIntentFilterForOculusDevices();
```

Android: 用于 BLE MIDI 的 CompanionDeviceManager

你可以使用 Android 的伴侣设备配对（Companion Device Pairing）来进行 BLE MIDI 设备连接。

启用方法：

- 添加脚本编译符号：`FEATURE_ANDROID_COMPANION_DEVICE`
- Unity 路径：`Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

注意：

- Meta Quest 设备可以使用此功能来查找/连接蓝牙 MIDI 设备。
- 根据 Android 版本或行为的不同，此功能可能需要请求位置权限。
- 在 Unity 6+（2023.1+）中，若 **Application Entry Point** 包含 **GameActivity**，后处理会将主 Activity 设为 `jp.kshoji.unity.midi.BleMidiUnityGamePlayerActivity`；仅选择 **Activity** 时使用 `jp.kshoji.unity.midi.BleMidiUnityPlayerActivity`。若同时启用两个入口，则会按类型将各 Unity 启动 Activity 改写为对应的 BLE MIDI Activity。

Nearby Connections MIDI (Google Nearby)

添加依赖包

在 Unity Package Manager 中：

- 点击 **+**
- 选择 **Add package from git URL...**
- 输入以下之一：
 - `git+https://github.com/kshoji/Nearby-Connections-for-Unity`
 - (SSH 方式) `ssh://git@github.com/kshoji/Nearby-Connections-for-Unity.git`

如果已经安装，请更新到最新版本。

启用脚本编译符号

添加：

- `ENABLE_NEARBY_CONNECTIONS`

Android 项目设置

将 Target API level 设置为 33 或更高：

- `Project Settings > Player > Identification > Target API Level`

用法概览

广播 (Advertising)：

- `MidiManager.Instance.StartNearbyAdvertising()`
- `MidiManager.Instance.StopNearbyAdvertising()`

发现 (Discovering)：

- `MidiManager.Instance.StartNearbyDiscovering()`
- `MidiManager.Instance.StopNearbyDiscovering()`

连接后，发送/接收 MIDI 数据的方式与普通 MIDI 相同。

Maestro / MPTK 集成 (可选)

该插件包含一个针对 **Maestro / MidiPlayerTK (MPTK)** 的可选集成层。

要启用它，请添加脚本编译符号：

- `FEATURE_USE_MPTK`

Unity 路径：

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

启用后的功能：

- 基于 MPTK 的**虚拟 MIDI 输出设备**（将 `MidiManager` 发送的消息路由到 MPTK 合成器）。
- 一个适配器，可以将 MPTK 播放器作为**虚拟 MIDI 输入源**（将事件注入到 `MidiManager`）。

注意：

- 仅当项目中存在 MPTK 资产时才启用此符号；否则，由于缺少 MPTK 类型，编译将失败。
- 另请参阅：[Maestro / MPTK 集成](#)

Unity 集成包（可选）

Timeline 和 Visual Scripting 集成需要**同时**安装对应的 Unity 包并添加脚本编译符号。

Animator 集成无需额外设置（包含在核心 asmdef `jp.kshoji.midi` 中）。

Unity 中设置符号的路径：

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

Timeline 集成

1. 通过 Package Manager 安装 **Timeline** :

- `com.unity.timeline`

2. 添加脚本编译符号 :

- `FEATURE_USE_TIMELINE`

启用后的功能 :

- Assembly Definition `jp.kshoji.midi.timeline` / `jp.kshoji.midi.timeline.editor`
- `MidiPlaybackTrack` / `MidiRecordTrack` 等 Timeline 联动组件
- 示例场景 `MidiTimelineIntegrationSampleScene`

注意 :

- 仅启用符号而不安装该包会导致 `Unity.Timeline` 引用错误。
- 禁用符号后, Timeline 集成代码将不参与编译 (核心插件仍可正常构建)。

Visual Scripting 集成

1. 通过 Package Manager 安装 **Visual Scripting** :

- `com.unity.visualscripting`

2. 添加脚本编译符号 :

- `FEATURE_USE_VISUALSCRIPTING`

启用后的功能 :

- Assembly Definition `jp.kshoji.midi.visualscripting`
- `MidiVisualScriptingBridge` 及 MIDI 自定义节点集
- 示例场景 `MidiVisualScriptingIntegrationSampleScene`

注意 :

- 仅启用符号而不安装该包会导致 `Unity.VisualScripting` 引用错误。
- 禁用符号后, Visual Scripting 集成代码将不参与编译。

Scriptable Audio Pipeline 集成（可选）

该插件包含针对 Unity 6.3+ [Scriptable Audio Pipeline](#) 的可选集成层。

要求：

- **Unity 6000.3 LTS** 或更高版本
- 支持的平台（非 WebGL）

要启用，请添加脚本编译符号：

- `FEATURE_SCRIPTABLE_AUDIO`

Unity 路径：

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

可选包（使用完整项目时已在 `Packages/manifest.json` 中）：

- `com.unity.burst`
- `com.unity.collections`

启用后的功能：

- Assembly Definition `jp.kshoji.midi.scriptableaudio`
- `ScriptableAudioBootstrap`、`MidiMetronomeGenerator`、`MidiDspClockBridge` 及 Pipe/DSP 时钟基础设施
- DSP 同步 SMF 播放：`MidiDspSequenceScheduler`、`MidiSequenceSynthGenerator`、`MidiDspSequenceBootstrap`
- DSP 同步 UMP 播放：`MidiDspUmpSequenceScheduler`、`UmpSequenceSynthGenerator`、`MidiDspUmpSequenceBootstrap`
- 示例场景 `ScriptableAudioMetronomeSampleScene`、`ScriptableAudioSequenceSampleScene`、`ScriptableAudioUmpSequenceSampleScene`

启用后快速开始：

1. **节拍器**：向 GameObject 添加 **Scriptable Audio Bootstrap**，或打开 `ScriptableAudioMetronomeSampleScene.unity`。
2. 进入 Play 模式 — 应能听到 120 BPM 节拍器点击声。

SMF 序列（DSP 同步播放）：

1. 添加 **Midi Dsp Sequence Bootstrap** 并分配 `MidiSequenceAsset`，或打开 `ScriptableAudioSequenceSampleScene.unity`。

2. 进入 Play 模式 — 音符在 Unity DSP 时钟上调度，由内置参考合成器播放。

UMP 序列（DSP 同步播放）：

1. 添加 **Midi Dsp Ump Sequence Bootstrap** 并分配 `UmpSequence`，或打开 `ScriptableAudioUmpSequenceSampleScene.unity`。
2. 进入 Play 模式 — UMP 剪辑音符在 Unity DSP 时钟上调度，由内置参考合成器播放。

注意：

- 未定义符号、Unity 6.2 及更早版本或 WebGL 上不会编译集成程序集；核心 `jp.kshoji.midi` 仍可正常构建。
- **SmfPlayer**（帧驱动、MIDI 设备输出）与 **MidiDspSequenceScheduler**（DSP 采样精度内置音频）是独立组件，请按用途选择其一。
- **UmpSequencer**（墙钟、专用线程）与 **MidiDspUmpSequenceScheduler**（DSP 同步内置音频）是独立组件；剪辑编辑/试播用前者，游戏内 BGM/循环/Seek 用后者。
- 编辑器诊断：**Window > MIDI > Validate Scriptable Audio Setup**（自动识别节拍器 / SMF / UMP Bootstrap）
- 另请参阅：[Scriptable Audio 集成](#)

Animator 集成

无需额外的包或脚本编译符号。

有关详细信息，请参阅 [Unity 生态系统集成](#)。

网络可选套件

网络 MIDI 同步需要添加脚本编译符号。

Unity 中设置符号的路径：

- Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols

网络 MIDI 同步

脚本编译符号：

- FEATURE_MIDI_NETWORK

启用后的功能：

- Assembly Definition jp.kshoji.midi.net
- MidiNetworkHub / MidiNetworkClient / MidiPlaybackSync
- 示例 MidiNetworkJamSampleScene

注意：

- UDP 端口（默认 55000–55002）必须在防火墙中放行。
- 禁用符号后，相关代码将不参与编译。

可选：Mirror / Netcode / WNet2 桥接

除 FEATURE_MIDI_NETWORK 外，仅在对应符号（及包门控）存在时编译可选桥接。框架本体 不会 随本仓库分发。

符号	说明
FEATURE_MIRROR	Mirror（MidiMirrorBridge）。另需 MIRROR（由 Mirror 定义）
FEATURE_NETCODE	Netcode for GameObjects（MidiNetcodeBridge）。MIDI_HAS_NETCODE 来自 UPM versionDefines
FEATURE_WSNET2	WSNet2（MidiWsn2Bridge）。存在 WNet2.Runtime.asmdef 时由 Editor 同步 MIDI_HAS_WSNET2

CI（BatchCompileBuilder）会附带 companion FEATURE_MIDI_NETWORK。无门控时排除桥接 asm, core net 仍可编译。

流程：安装框架 → 符号 → 组件接线 → 示例。详见 [类型套件](#)、[集成](#)、[Networking 集成](#)。

基于 UDP 的 `MidiNetworkHub` / `MidiNetworkClient` 无需上述符号即可使用。

横向基础可选套件

Input System 桥接需要添加脚本编译符号。

MidiClockSync / 和弦与音阶判定无需符号（核心 `jp.kshoji.midi`）。

Unity 中设置符号的路径：

- `Project Settings > Player > Other Settings > Script Compilation > Scripting Define Symbols`

Input System 桥接

1. 通过 Package Manager 添加 `com.unity.inputsystem`（若已随 `Packages/manifest.json` 一同提供则无需添加）。
2. 添加脚本编译符号：
 - `FEATURE_INPUT_SYSTEM`

启用后的功能：

- Assembly Definition `jp.kshoji.midi.inputsystem`
- `MidiInputSystemBridge` / `InputSystemToMidiBridge` / `MidiSyntheticDevice`
- 示例 `InputSystemBridgeSampleScene`

注意：

- 禁用符号后，Input System 集成代码将不参与编译（核心插件仍可正常构建）。

MidiClockSync / 和弦与音阶判定

无需额外符号。包含在核心 `asmdef`（`jp.kshoji.midi`）中。

有关详细信息，请参阅 [类型套件 — 横向基础](#)。

Chunity (Chuck) 集成（可选）

本插件**不捆绑 Chunity 运行时**。仅当使用者另行安装并启用符号时，联动代码才会参与编译。

1. 将 [Chunity](#) 安装到项目中。
2. 将 `Assets/MIDI/Scripts/Integrations/Chunity/Optional/Chunity.Runtime.asmdef.example` 作为 `Chunity.Runtime.asmdef` 复制到 Chunity 的 Scripts 根目录（例如 `Assets/Chunity/Scripts/Chunity.Runtime.asmdef`）。详情参见 [Chunity Optional 补丁说明](#)。
3. 添加脚本编译符号：
 - `FEATURE_CHUNITY`

启用后的功能：

- Assembly Definition `jp.kshoji.midi.chunity`
- `MidiChuckBridge` / `MidiChuckPatchHost` / `MidiChuckMapping` / Phase 2–3 组件
- 示例 `ChunityBridgeSampleScene` / `ChunityWorkflowsSampleScene` / `ChunityPresetsSampleScene`
- （可选）`FEATURE_USE_TIMELINE` → `jp.kshoji.midi.chunity.timeline`
- （可选）`FEATURE_USE_VISUALSCRIPTING` → `jp.kshoji.midi.chunity.visualscripting`

Scriptable Audio Generator（附加可选）

将 Chuck 的音频输出从 `OnAudioFilterRead` 替换为 Unity 6.3+ 的 Scriptable Generator（控制路径不变）。

1. 按照 [Chunity Optional 补丁说明](#) 中的步骤，为 Chunity 本体应用 `useBuiltInAudioFilter` 补丁。
2. 通过 Package Manager 安装 `** com.unity.collections **`（asmdef 引用 `Unity.Collections`；`NativeArray` / `FixedString128Bytes` 需要它）。
3. 添加脚本编译符号：
 - `FEATURE_CHUNITY_SCRIPTABLE_AUDIO`（同时需要 `FEATURE_CHUNITY`）

启用后的功能：

- Assembly Definition `jp.kshoji.midi.chunity.scriptableaudio`
- `ChuckMainGeneratorDriver` / `ChuckSubGeneratorDriver`
- 示例 `ChunityGeneratorWorkflowSampleScene`

注意：在 WebGL 及不受支持的环境中，将回退到传统的 `FilterRead` / `WebChuck`。节拍器等本体 Scriptable Audio 集成使用的是另一个符号 `FEATURE_SCRIPTABLE_AUDIO`。

注意：

- 仅启用符号而未导入 Chunity（或未放置 `Chunity.Runtime` asmdef）时，将出现类型解析错误（与 MPTK 相同）。
- 禁用符号后，集成代码将不参与编译（核心插件仍可正常构建）。
- 面向用户的说明：[Unity 生态系统集成 — Chunity](#)
- 设置详情：[Chunity 集成](#)